

Informe de la Guia Practica de Laboratorio N. 09

Capacidades nativas con expect actual en PharmaMobile

Dato	Detalle
Estudiante	Luciana Gutierrez
Asignatura	Desarrollo de Aplicaciones Moviles
Ciclo y semestre	VI 2026-2
Fecha	6 de octubre de 2026
Proyecto	PharmaMobile integrado con PharmaSoft
Rama	feature/expect-actual-Gutierrez

Implementacion y verificacion de formato monetario por plataforma, compartir nativo e inyeccion con Koin, sin modificar el CRUD existente.

1. Resumen del resultado

PharmaMobile cumple la implementación técnica solicitada para la sesión 09. El precio se transforma a moneda peruana mediante expect/actual y la acción de compartir se resuelve con un contrato común y una implementación nativa por plataforma. Android fue compilado, probado y ejecutado en un emulador real. El código de iosMain fue compilado, pero la ejecución de iOS permanece pendiente porque el entorno disponible es Windows y no cuenta con Xcode.

Verificación	Resultado real
Pruebas Android host	34 pruebas, 0 fallos, 0 errores
APK Android	BUILD SUCCESSFUL
iosMain	Compilación Kotlin exitosa; ejecución no realizada
Backend	PharmaSoft respondió HTTP 200
Ejecución Android	Precio y selector de compartir verificados
Imports prohibidos	Ninguno en presentation, presentación o domain común

2. Estado inicial y estrategia Git

El proyecto ya contenía el CRUD REST de productos, Ktor Client, DTO, mappers, repositorio, Koin, ProductoViewModel y la interfaz Compose. También existían PlatformModule para Android e iOS y los puentes Swift del framework Shared. No existían el formateador expect/actual, ProductoUi con precio formateado, Compartidor ni el botón de compartir.

La rama develop estaba 20 commits detrás de la rama con el CRUD de la sesión 8. Para evitar perder trabajo, feature/expect-actual-Gutierrez se creó desde feature/crud-productos-Gutierrez. No se cambió el remoto, no se reescribió historial y no se realizó push durante la implementación.

3. Formato monetario con expect actual

3.1 Declaración común

```
package pe.edu.upeu.pharmamobilee.platform

expect fun formatearSoles(valor: Double): String
```

3.2 Implementación Android

```
actual fun formatearSoles(valor: Double): String =
    NumberFormat.getCurrencyInstance(Locale("es", "PE")).format(valor)
```

3.3 Implementación iOS

```
actual fun formatearSoles(valor: Double): String {
    val formateador = NSNumberFormatter().apply {
        numberStyle = NSNumberFormatter.CurrencyStyle
        locale = NSLocale("es_PE")
    }
```

```

    }
    return formateador.stringFromNumber(NSNumber(valor)) ?: "S/ $valor"
}

```

3.4 Aplicacion en presentacion

Producto conserva precio como Double. Producto.toUi() invoca formatearSoles y entrega a Compose un ProductoUi con el precio listo para visualizar. El composable no realiza redondeos, conversiones de moneda ni llamadas a APIs nativas.

```

fun Producto.toUi(): ProductoUi = ProductoUi(
    producto = this,
    precio = formatearSoles(precio)
)

```

4. Punto de control del compilador

Antes de crear los actual se ejecuto :androidApp:assembleDebug con unicamente la declaracion expect. Gradle produjo el siguiente error real, confirmando que el compilador exige la implementacion de cada plataforma.

```

> Task :shared:compileAndroidMain FAILED
e: ../commonMain/.../platform/Formato.kt:3:1
Expected formatearSoles has no actual declaration in module <commonMain> for JVM

BUILD FAILED

```

Despues de incorporar Formato.android.kt y Formato.ios.kt, la compilacion Android y la compilacion del target iosSimulatorArm64 finalizaron correctamente.

5. Compartir producto por plataforma

5.1 Contrato y texto comun

```

interface Compartidor {
    fun compartir(texto: String)
}

fun Producto.comoTextoParaCompartir(): String =
    "$nombre - ${formatearSoles(precio)} - Stock: $stock"

```

El contrato no importa Android ni UIKit. La composicion del mensaje permanece en commonMain y se prueba con un doble de Compartidor.

Plataforma	Implementacion	API nativa
Android	CompartidorAndroid	ACTION_SEND, text/plain, EXTRA_TEXT, chooser y NEW_TASK
iOS	CompartidorIos	UIActivityViewController y presentacion desde rootViewController

5.2 Inyeccion y accion de presentacion

Cada PlatformModule registra single<Compartidor> con la implementacion de su plataforma. ProductoViewModel recibe el contrato por constructor y expone compartir(producto). La tarjeta Compose solo invoca esa accion comun.

6. Evidencias reales de Android

Las dos figuras siguientes fueron obtenidas directamente del emulador Android después de instalar el APK generado en esta rama. PharmaSoft se encontraba respondiendo en el puerto 8080.

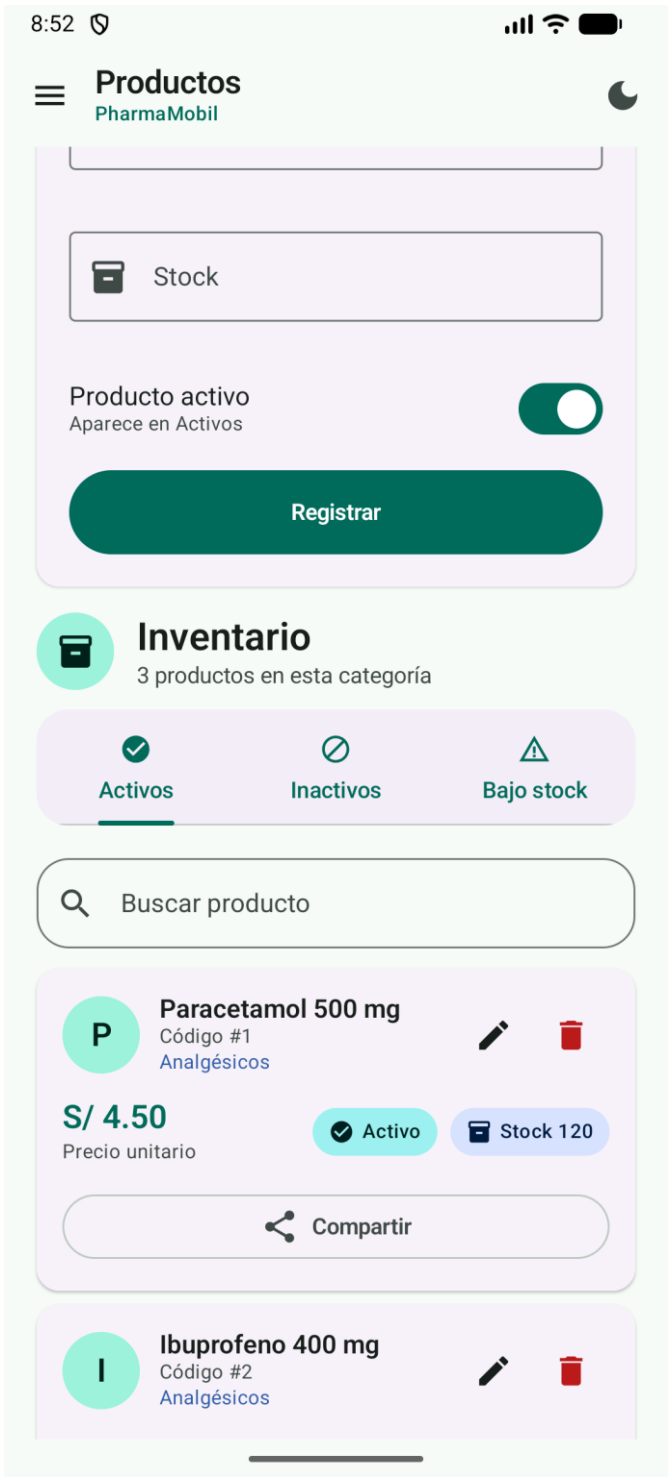


Figura 1. Listado Android con precio S/ 4.50 y boton Compartir.

7. Evidencia del selector y el texto

Al pulsar Compartir en Paracetamol 500 mg se abrió el selector nativo de Android. La vista previa confirma que el texto contiene nombre, precio monetario y stock.

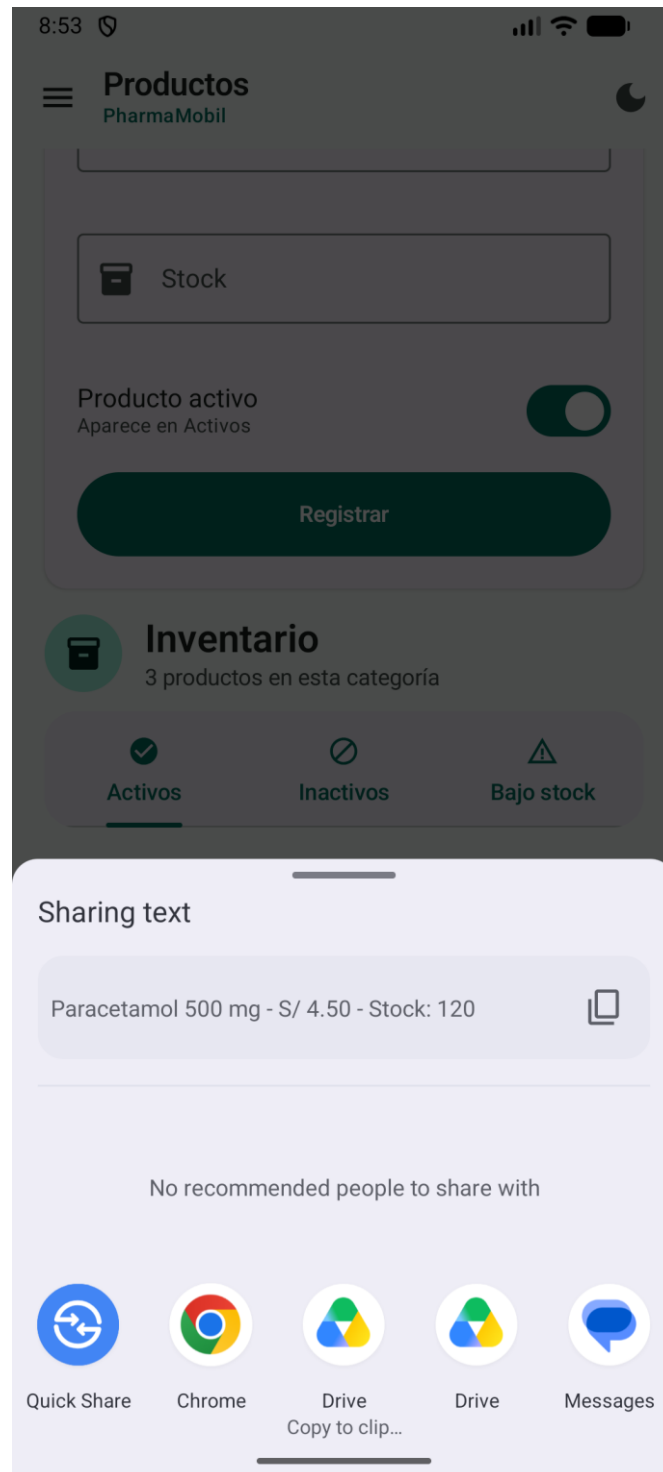


Figura 2. Selector Android con 'Paracetamol 500 mg - S/ 4.50 - Stock: 120'.

8. Arquitectura y separacion por plataforma

Capa o source set	Responsabilidad implementada
commonMain platform	Declaracion expect de formatearSoles
androidMain platform	NumberFormat y CompartidorAndroid
iosMain platform	NSNumberFormatter y Compartidorlos
commonMain domain	Contrato Compartidor y texto para compartir
commonMain presentation	ProductoUi, mapper, ViewModel y accion comun
commonMain Compose	Precio formateado y boton Compartir
di	Seleccion de implementacion mediante PlatformModule

La auditoria con busqueda global no encontro importaciones android.* ni platform.UIKit.* dentro de presentation, presentacion o domain en commonMain.

9. Interoperabilidad Kotlin Swift

El modulo shared se exporta como el framework Shared. iOSApp.swift importa ese framework e inicializa Koin mediante KoinlosKt.initKoinlos(). ContentView.swift obtiene el controlador Compose mediante MainViewControllerKt.MainViewController(). Los sufijos Kt corresponden a las clases generadas para funciones Kotlin de nivel superior.

10. Pruebas y compilaciones

Comando o control	Resultado
:shared:testAndroidHostTest	34 pruebas aprobadas
:androidApp:assembleDebug	BUILD SUCCESSFUL
:shared:compileKotlinIosSimulatorArm64	BUILD SUCCESSFUL
Ejecucion Android	Listado y selector verificados en Medium_Phone
Ejecucion iOS	PENDIENTE DE MAC/XCODE

11. Commits distribuidos

Hash	Descripcion
dc2421d	feat: agrega formato monetario con expect actual
be1911c	feat: implementa compartidor nativo por plataforma
f70c47d	feat: integra compartidor con Koin y productos
0cc6d53	docs: documenta capacidades nativas de sesion 09

La rama utilizada es feature/expect-actual-Gutierrez. En el momento de esta verificacion la rama no tenia upstream y no se habia realizado push.

12. Evidencias pendientes en macOS

- Ejecutar iosApp en un simulador desde Xcode.
- Capturar el listado con el precio producido por NSNumberFormatter.
- Pulsar Compartir y capturar UIActivityViewController.
- Confirmar visualmente que el texto contiene nombre, precio y stock.

Estas evidencias no se sustituyeron con montajes ni capturas de otra plataforma. El informe diferencia implementacion, compilacion y ejecucion real.

13. Lista de cotejo de los 10 criterios

N.	Criterio	Estado	Evidencia
1	expect en commonMain y actual en Android e iOS	CUMPLE	Tres archivos presentes y ambos targets compilados
2	Mismo paquete y firma	CUMPLE	pe.edu.upeu.pharmamobilee.platform
3	Precio como moneda en ambas plataformas	PENDIENTE iOS	Android evidenciado; iOS requiere simulador
4	Formato aplicado en presentacion	CUMPLE	Producto.toUi aplica formatearSoles
5	Compartidor en domain sin plataforma	CUMPLE	Contrato comun sin imports nativos
6	Implementacion por plataforma	CUMPLE TECNICO	Android ejecutado; iOS compilado
7	PlatformModule registra Compartidor	PENDIENTE iOS	Android resuelto en ejecucion; iOS sin ejecucion
8	Boton funciona por plataforma	PENDIENTE iOS	Android evidenciado; iOS requiere Xcode
9	Presentation sin Android ni UIKit	CUMPLE	Busqueda global sin resultados
10	Capturas equivalentes Android e iOS	PENDIENTE MAC/XCODE	No se fabricaron capturas iOS

14. Conclusion

La Guía Práctica 09 queda implementada sin rehacer el CRUD de la sesión 8 y sin introducir dependencias nativas en el código común de presentación. Android cuenta con evidencia funcional completa. La implementación iOS compila, pero su evidencia visual debe completarse en macOS con Xcode para cerrar los criterios multiplataforma.